Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 2.5

Obtained Marks: _____

Artificial Neural Networks

(Lab)

Task # 01

Submitted To: Mr. Hassaan Mujtaba

Student Name: Taimur Sultan

Registration Number: 24108145

Code # 1:

```python
#  Dataset 1 Pesticides Data -> Predict pesticide use "Value" (tonnes)
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

# 1. Load the data
df = pd.read_csv("pesticides.csv")
print(df.head())

# 2. Choose input columns (X) and target column (y)
# "Area" (country) is text, so get_dummies turns it into 0/1 columns
X = pd.get_dummies(df[["Area", "Year"]])
y = df["Value"]

# 3. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# 5. Check how good the model is
y_pred = model.predict(X_test)
print("R2 Score:", r2_score(y_test, y_pred))
print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))


# 6. Prediction function (used by both Gradio and Flask)
countries = sorted(df["Area"].unique())

def predict(country, year):
    data = pd.DataFrame(0, index=[0], columns=X.columns)   # one row of zeros
    data["Year"] = year
    data["Area_" + country] = 1                            # mark the chosen country
    result = model.predict(data)[0]
    return round(result, 2)


# ===== 7. Gradio Interface =====
import gradio as gr

app_gradio = gr.Interface(
    fn=predict,
    inputs=[
```

```
app_gradio = gr.Interface(
    fn=predict,
    inputs=[
        gr.Dropdown(countries, label="Country"),
        gr.Number(label="Year"),
    ],
    outputs=gr.Number(label="Predicted Pesticide Use (tonnes)"),
    title="Pesticide Use Predictor (Linear Regression)",
)

app_gradio.launch()


# ===== 8. Flask Interface =====

from flask import Flask, request, render_template_string

app = Flask(__name__)

html = """
<h2>Pesticide Use Predictor (Linear Regression)</h2>
<form method="post">
    Country:
    <select name="country">
        {% for c in countries %}
        <option>{{ c }}</option>
        {% endfor %}
    </select><br><br>
    Year: <input name="year" required><br><br>
    <button type="submit">Predict</button>
</form>
<h3>{{ result }}</h3>
"""

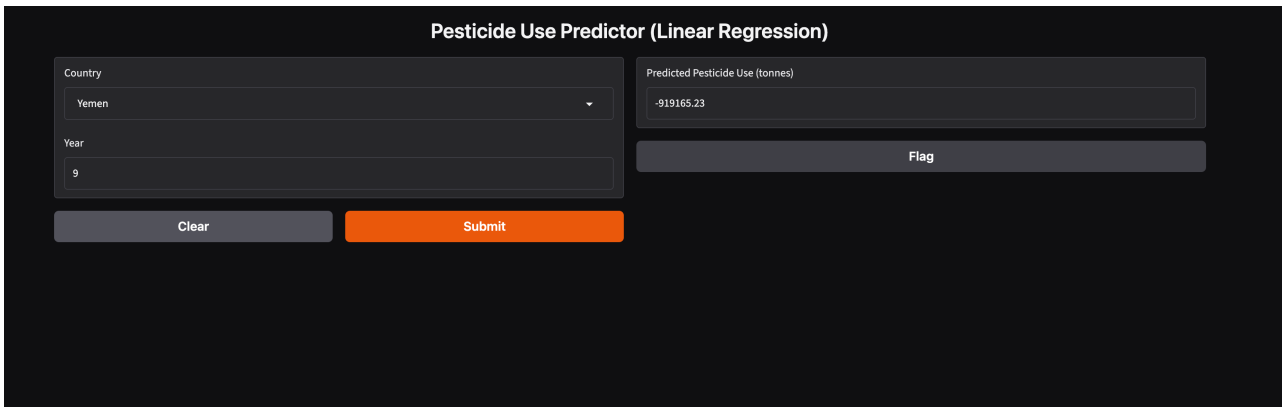
@app.route("/", methods=["GET", "POST"])
def home():
    result = ""
    if request.method == "POST":
        country = request.form["country"]
        year = int(request.form["year"])
        result = "Predicted Pesticide Use: " + str(predict(country, year)) + " tonnes"
    return render_template_string(html, result=result, countries=countries)

app.run(port=5002)
```

Output:

```
            Domain     Area Element                Item  Year  \
 0  Pesticides Use  Albania     Use  Pesticides (total)  1990
 1  Pesticides Use  Albania     Use  Pesticides (total)  1991
 2  Pesticides Use  Albania     Use  Pesticides (total)  1992
 3  Pesticides Use  Albania     Use  Pesticides (total)  1993
 4  Pesticides Use  Albania     Use  Pesticides (total)  1994

                          Unit  Value
 0  tonnes of active ingredients  121.0
 1  tonnes of active ingredients  121.0
 2  tonnes of active ingredients  121.0
 3  tonnes of active ingredients  121.0
 4  tonnes of active ingredients  201.0
 R2 Score: 0.9345056989283204
 Mean Absolute Error: 7492.419186478035
 * Running on local URL:  http://127.0.0.1:7860
 * To create a public link, set `share=True` in `launch()`.
```



Code # 2:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

# 1. Load the data
df = pd.read_csv("student-por.csv")
print(df.head())

# 2. Choose input columns (X) and target column (y)
X = df[["G1", "G2", "studytime", "failures", "absences"]]
y = df["G3"]

# 3. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# 5. Check how good the model is
y_pred = model.predict(X_test)
print("R2 Score:", r2_score(y_test, y_pred))
print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))


# 6. Prediction function (used by both Gradio and Flask)
def predict(g1, g2, studytime, failures, absences):
    data = pd.DataFrame([[g1, g2, studytime, failures, absences]], columns=X.columns)
    result = model.predict(data)[0]
    return round(result, 2)


# ===== 7. Gradio Interface =====
import gradio as gr

app_gradio = gr.Interface(
    fn=predict,
    inputs=[
        gr.Number(label="G1 (First period grade, 0-20)"),
        gr.Number(label="G2 (Second period grade, 0-20)"),
        gr.Number(label="Study Time (1-4)"),
        gr.Number(label="Past Failures (0-3)"),
        gr.Number(label="Absences"),
    ],
```

```
    outputs=gr.Number(label="Predicted Final Grade (G3)"),
    title="Student Final Grade Predictor (Linear Regression)",
)

app_gradio.launch()


# ===== 8. Flask Interface =====

from flask import Flask, request, render_template_string

app = Flask(__name__)

html = """
<h2>Student Final Grade Predictor (Linear Regression)</h2>
<form method="post">
    G1 (First period grade, 0-20): <input name="g1" required><br><br>
    G2 (Second period grade, 0-20): <input name="g2" required><br><br>
    Study Time (1-4): <input name="studytime" required><br><br>
    Past Failures (0-3): <input name="failures" required><br><br>
    Absences: <input name="absences" required><br><br>
    <button type="submit">Predict</button>
</form>
<h3>{{ result }}</h3>
"""

@app.route("/", methods=["GET", "POST"])
def home():
    result = ""
    if request.method == "POST":
        g1 = float(request.form["g1"])
        g2 = float(request.form["g2"])
        studytime = float(request.form["studytime"])
        failures = float(request.form["failures"])
        absences = float(request.form["absences"])
        result = "Predicted Final Grade (G3): " + str(predict(g1, g2, studytime, failures, absences))
    return render_template_string(html, result=result)

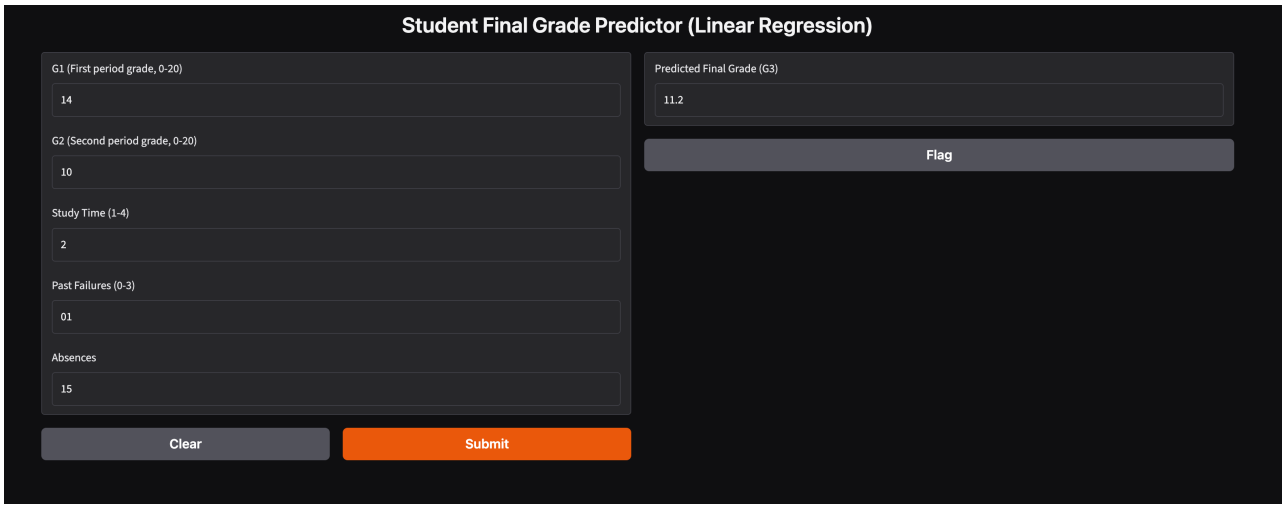
app.run(port=5003)
```

Output:

```
  school sex  age address famsize Pstatus  Medu  Fedu     Mjob      Fjob  ...  \
0     GP   F   18       U     GT3       A     4     4  at_home   teacher  ...
1     GP   F   17       U     GT3       T     1     1  at_home     other  ...
2     GP   F   15       U     LE3       T     1     1  at_home     other  ...
3     GP   F   15       U     GT3       T     4     2   health  services  ...
4     GP   F   16       U     GT3       T     3     3    other     other  ...

  famrel freetime  goout  Dalc  Walc health absences  G1  G2  G3
0      4        3      4     1     1      3        4   0  11  11
1      5        3      3     1     1      3        2   9  11  11
2      4        3      2     2     3      3        6  12  13  12
3      3        2      2     1     1      5        0  14  14  14
4      4        3      2     1     2      5        0  11  13  13

[5 rows x 33 columns]
R2 Score: 0.863263569485066
Mean Absolute Error: 0.7462407529568897
* Running on local URL:  http://127.0.0.1:7860
* To create a public link, set `share=True` in `launch()`.
```



GITHUB LINK:
https://github.com/taimurkhaneth